

Tarea2

2026-02-13

Cargar dataset MovieLens

```
## user_id item_id rating timestamp
## 1 196 242 3 881250949
## 2 186 302 3 891717742
## 3 22 377 1 878887116
## 4 244 51 2 880606923
## 5 166 346 1 886397596
## 6 298 474 4 884182806
```

Dimensiones del dataset

```
## [1] 100000 4
## [1] 943
## [1] 1682
```

Crear matriz usuario $\times$ película

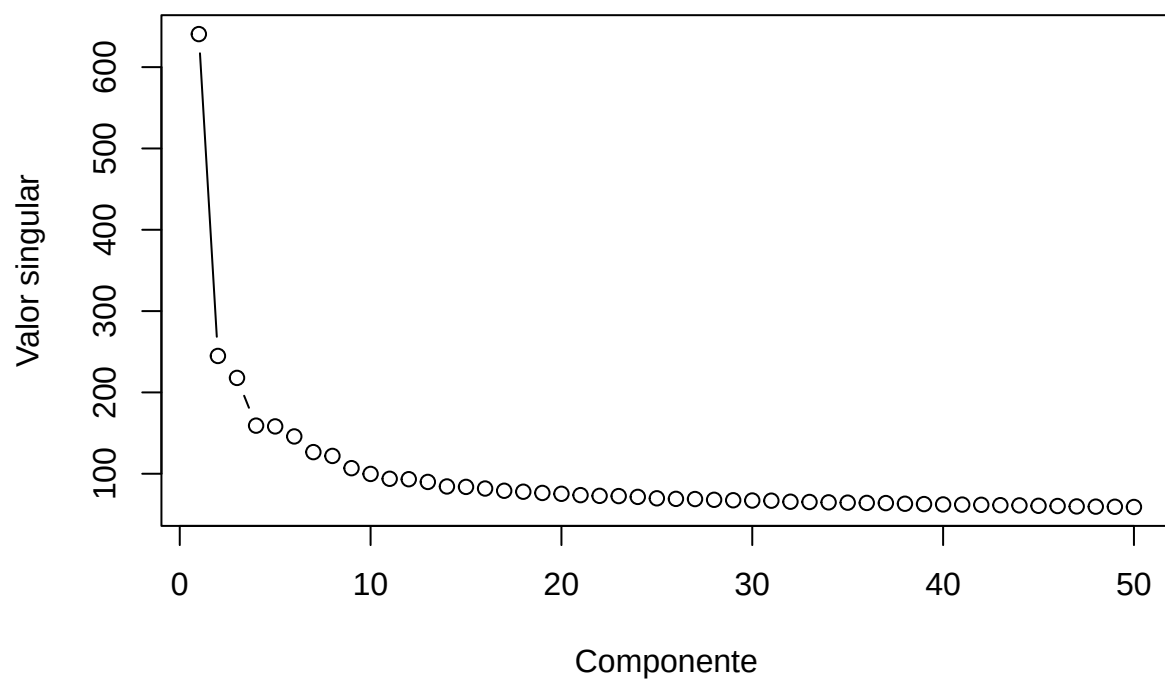
```
## [1] 943 1682
```

Aplicar SVD

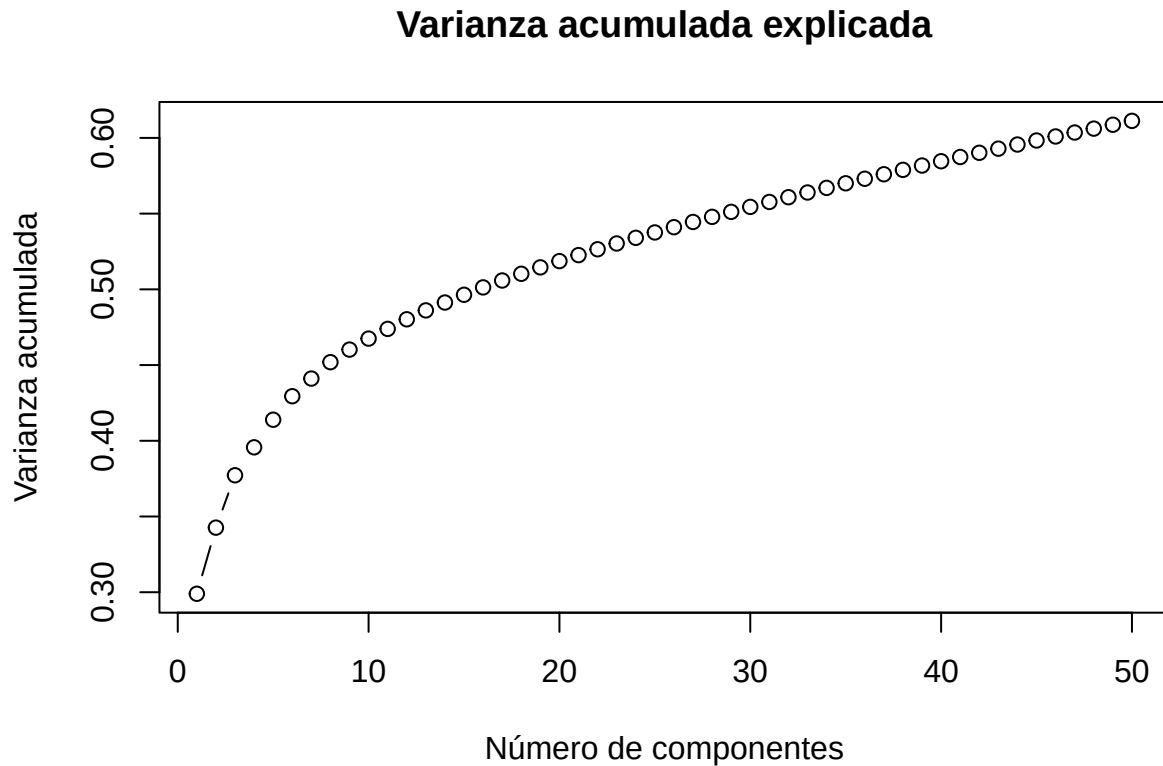
```
## [1] 640.6336 244.8363 217.8462 159.1536 158.2119 145.8726
```

Graficar valores singulares

Primeros 50 valores singulares



Varianza explicada



Reducción a k dimensiones (ejemplo $k = 20$)

```
## [1] 943 1682
```

Interpretación de resultados

Al aplicar SVD sobre la matriz usuario-película, se obtuvo una descomposición en tres matrices (U, D y V), donde los valores singulares representan la importancia de cada componente latente.

El gráfico de valores singulares muestra que los primeros componentes concentran la mayor parte de la información del sistema. La gráfica de varianza acumulada indica que con un número reducido de componentes (aproximadamente 15–25) se puede explicar un alto porcentaje de la variabilidad total.

Esto demuestra que el sistema de calificaciones puede representarse en un espacio de menor dimensión, lo cual es fundamental en sistemas de recomendación, ya que permite identificar factores latentes como:

- Preferencias de género
- Tendencias de los usuarios

- Popularidad implícita de películas

La reducción a $k = 20$ componentes permite reconstruir una aproximación de la matriz original, conservando la mayor parte de la información relevante, pero reduciendo considerablemente la dimensionalidad.

2.1 SVD (Singular Value Decomposition) *Descripción teórica*

La Descomposición en Valores Singulares (SVD) es una técnica de álgebra lineal que permite factorizar una matriz A en el producto de tres matrices:

$$A = UDV^T$$

donde:

- U contiene los vectores singulares izquierdos.
- D es una matriz diagonal con los valores singulares.
- V contiene los vectores singulares derechos.

El objetivo principal de SVD dentro del aprendizaje no supervisado es reducir dimensionalidad y extraer factores latentes que expliquen la estructura interna de los datos.

A diferencia de PCA, que se basa en la matriz de covarianza, SVD puede aplicarse directamente sobre cualquier matriz rectangular, incluso si no está centrada.

Características principales

- Permite reducción de dimensionalidad.
- Identifica patrones latentes.
- No requiere variable objetivo.
- Es base matemática de PCA.
- Se utiliza ampliamente en sistemas de recomendación.

Diferencias con PCA

- PCA trabaja sobre la matriz de covarianza.
- SVD trabaja directamente sobre la matriz original.
- PCA requiere centrar los datos.
- SVD no necesariamente requiere centrado.
- En sistemas de recomendación, SVD es más común.

Usos y aplicaciones

Usos y aplicaciones

SVD se utiliza principalmente en:

1. **Sistemas de recomendación** • Identificación de preferencias latentes de usuarios.
- Predicción de calificaciones faltantes.
2. **Reducción de dimensionalidad** • Compresión de información.
- Visualización de datos en espacios de menor dimensión.
3. **Procesamiento de texto** • Análisis semántico latente (LSA).
4. **Compresión de imágenes** • Eliminación de ruido y almacenamiento eficiente.

SVD es adecuado para estos problemas porque permite representar grandes matrices en un espacio reducido conservando la mayor parte de la información.

Dataset utilizado

Se utilizó el dataset MovieLens 100K, que contiene 100,000 calificaciones de películas realizadas por 943 usuarios sobre 1682 películas.

El dataset incluye:

- user_id
- item_id
- rating
- timestamp

Para aplicar SVD, se construyó una matriz usuario $\times$ película donde los valores representan las calificaciones otorgadas.

2.2 t-SNE

Cargar dataset Breast Cancer

```
##      id diagnosis radius_mean texture_mean perimeter_mean area_mean
## 1  842302      M      17.99      10.38      122.80      1001.0
## 2  842517      M      20.57      17.77      132.90      1326.0
## 3  84300903     M      19.69      21.25      130.00      1203.0
## 4  84348301     M      11.42      20.38       77.58       386.1
## 5  84358402     M      20.29      14.34      135.10      1297.0
## 6  843786      M      12.45      15.70       82.57       477.1
##      smoothness_mean compactness_mean concavity_mean concave.points_mean
## 1      0.11840      0.27760      0.3001      0.14710
## 2      0.08474      0.07864      0.0869      0.07017
## 3      0.10960      0.15990      0.1974      0.12790
## 4      0.14250      0.28390      0.2414      0.10520
## 5      0.10030      0.13280      0.1980      0.10430
## 6      0.12780      0.17000      0.1578      0.08089
##      symmetry_mean fractal_dimension_mean radius_se texture_se perimeter_se
## 1      0.2419      0.07871      1.0950      0.9053      8.589
## 2      0.1812      0.05667      0.5435      0.7339      3.398
## 3      0.2069      0.05999      0.7456      0.7869      4.585
## 4      0.2597      0.09744      0.4956      1.1560      3.445
## 5      0.1809      0.05883      0.7572      0.7813      5.438
## 6      0.2087      0.07613      0.3345      0.8902      2.217
##      area_se smoothness_se compactness_se concavity_se concave.points_se
## 1  153.40      0.006399      0.04904      0.05373      0.01587
## 2   74.08      0.005225      0.01308      0.01860      0.01340
## 3   94.03      0.006150      0.04006      0.03832      0.02058
## 4   27.23      0.009110      0.07458      0.05661      0.01867
## 5   94.44      0.011490      0.02461      0.05688      0.01885
## 6   27.19      0.007510      0.03345      0.03672      0.01137
##      symmetry_se fractal_dimension_se radius_worst texture_worst perimeter_worst
## 1   0.03003      0.006193      25.38      17.33      184.60
## 2   0.01389      0.003532      24.99      23.41      158.80
## 3   0.02250      0.004571      23.57      25.53      152.50
## 4   0.05963      0.009208      14.91      26.50      98.87
## 5   0.01756      0.005115      22.54      16.67      152.20
## 6   0.02165      0.005082      15.47      23.75      103.40
##      area_worst smoothness_worst compactness_worst concavity_worst
## 1  2019.0      0.1622      0.6656      0.7119
## 2  1956.0      0.1238      0.1866      0.2416
## 3  1709.0      0.1444      0.4245      0.4504
## 4   567.7      0.2098      0.8663      0.6869
## 5  1575.0      0.1374      0.2050      0.4000
## 6   741.6      0.1791      0.5249      0.5355
##      concave.points_worst symmetry_worst fractal_dimension_worst X
## 1      0.2654      0.4601      0.11890 NA
## 2      0.1860      0.2750      0.08902 NA
## 3      0.2430      0.3613      0.08758 NA
## 4      0.2575      0.6638      0.17300 NA
## 5      0.1625      0.2364      0.07678 NA
## 6      0.1741      0.3985      0.12440 NA
```

```
## [1] 569 33
```

Limpieza de datos

Quitamos columnas innecesarias:

- id
- X (si aparece una columna vacía al final)

```
## [1] 569 30
```

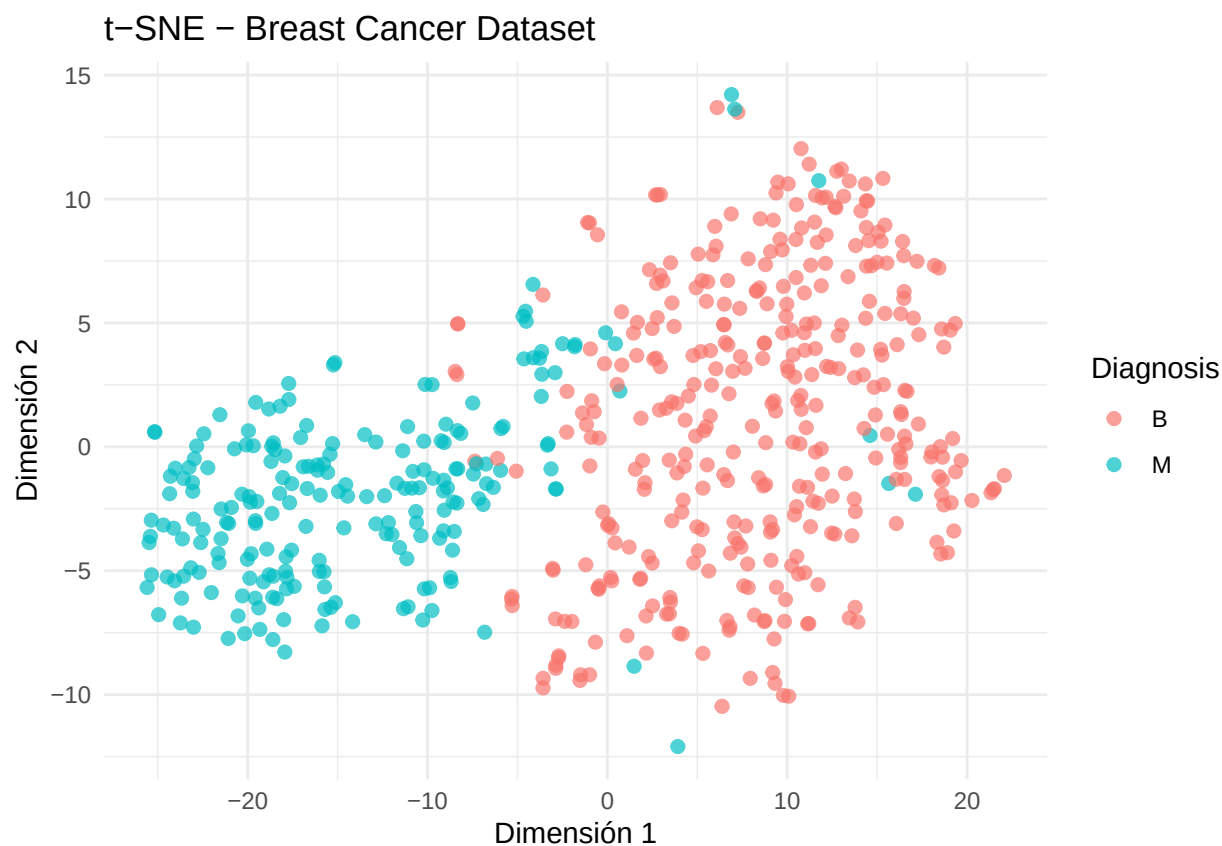
Escalar los datos

Aplicar t-SNE

```
## Performing PCA
## Read the 569 x 30 data matrix successfully!
## Using no_dims = 2, perplexity = 30.000000, and theta = 0.500000
## Computing input similarities...
## Building tree...
## Done in 0.02 seconds (sparsity = 0.224116)!
## Learning embedding...
## Iteration 50: error is 62.155687 (50 iterations in 0.04 seconds)
## Iteration 100: error is 59.863724 (50 iterations in 0.03 seconds)
## Iteration 150: error is 59.868511 (50 iterations in 0.03 seconds)
## Iteration 200: error is 59.873753 (50 iterations in 0.03 seconds)
## Iteration 250: error is 59.865082 (50 iterations in 0.03 seconds)
## Iteration 300: error is 1.105521 (50 iterations in 0.03 seconds)
## Iteration 350: error is 1.013762 (50 iterations in 0.03 seconds)
## Iteration 400: error is 0.991826 (50 iterations in 0.03 seconds)
## Iteration 450: error is 0.982656 (50 iterations in 0.03 seconds)
## Iteration 500: error is 0.974774 (50 iterations in 0.03 seconds)
## Fitting performed in 0.29 seconds.
```

```
##          Dim1      Dim2 Diagnosis
## 1 -24.475358 -5.2511391          M
## 2 -17.930739 -0.3682355          M
## 3 -21.169202 -3.0476322          M
## 4  -8.687985 -5.4336150          M
## 5 -17.561518 -4.1676592          M
## 6  -9.093245 -1.3473411          M
```

Graficar resultados



Interpretación de resultados

El algoritmo t-SNE permitió reducir las 30 variables originales del dataset a un espacio bidimensional.

En la visualización obtenida se observa una clara separación entre las observaciones diagnosticadas como malignas (M) y benignas (B). Esto indica que el dataset posee una estructura interna bien definida y que las variables contienen suficiente información para diferenciar ambas clases.

A diferencia de PCA, t-SNE no busca maximizar varianza global, sino preservar la estructura local de los datos, lo que explica la formación de grupos bien definidos.

El resultado confirma que t-SNE es especialmente útil para:

- Visualización de datos de alta dimensión
- Identificación de agrupamientos naturales
- Exploración de estructuras complejas no lineales

Sin embargo, t-SNE no es un método de clasificación, sino una herramienta de visualización y reducción de dimensionalidad no lineal.

2.2 t-SNE (t-Distributed Stochastic Neighbor Embedding)

Descripción teórica

t-SNE es un algoritmo de reducción de dimensionalidad no lineal diseñado principalmente para visualización de datos de alta dimensión.

Su objetivo es preservar las relaciones de vecindad entre observaciones. Para ello:

1. Convierte las distancias entre puntos en probabilidades.
2. Minimiza la diferencia entre las distribuciones de alta dimensión y baja dimensión usando divergencia KL.

A diferencia de PCA y SVD, t-SNE:

- No es lineal.
- No genera componentes interpretables.
- Está optimizado para visualización.

Características principales

- Preserva estructura local.
- Funciona bien con datos complejos.
- Sensible a parámetros como perplexity.
- No mantiene estructura global con precisión.

USOS Y APLICACIONES

Usos y aplicaciones

t-SNE se utiliza en:

1. Visualización de datos biomédicos.
2. Análisis de imágenes.
3. Procesamiento de texto y embeddings.
4. Exploración de clusters en datasets complejos.

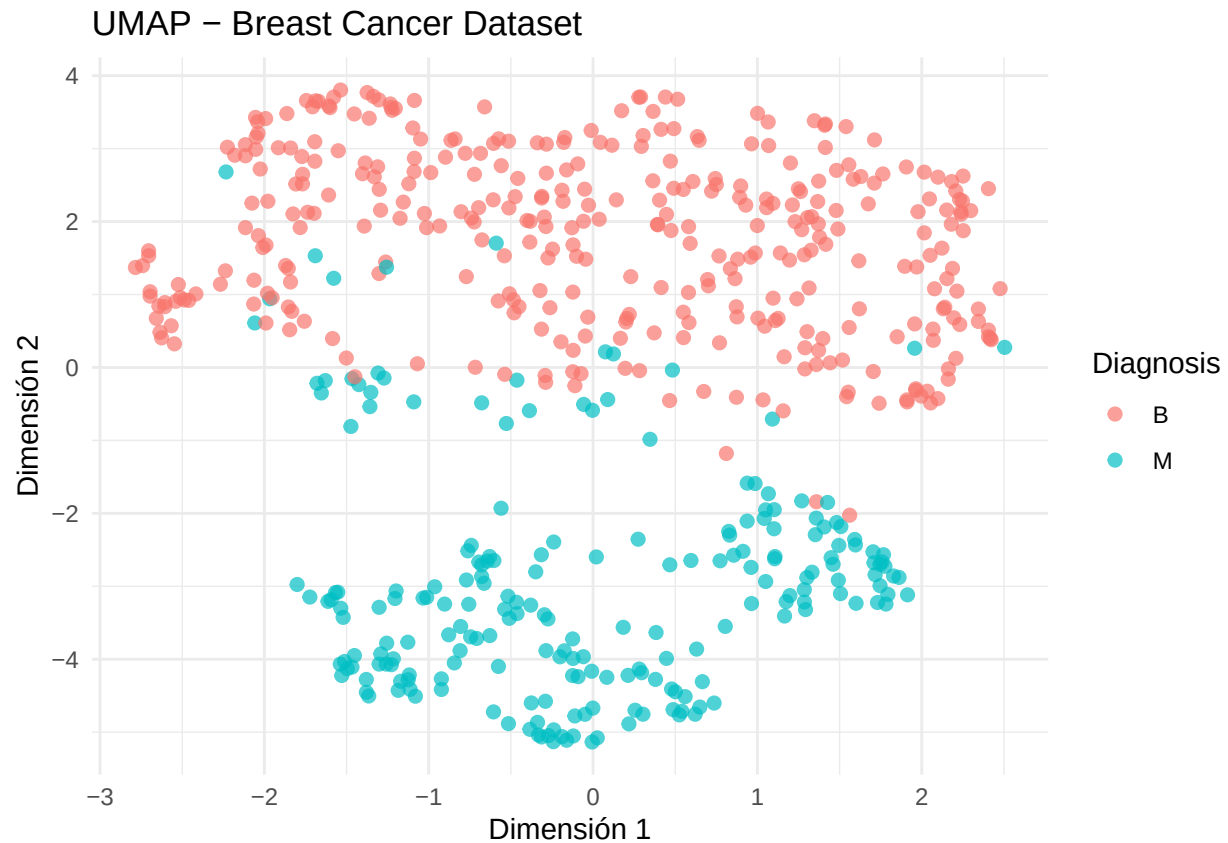
Es especialmente útil cuando se desea identificar agrupamientos naturales en datos con muchas variables.

2.3 UMAP

Aplicar UMAP

```
##          Dim1      Dim2 Diagnosis
## 1  0.5601494 -4.512411         M
## 2 -0.9633114 -3.006444         M
## 3  0.0837107 -4.247522         M
## 4  1.7310826 -3.221959         M
## 5 -1.2558066 -3.779976         M
## 6  1.7461153 -2.699165         M
```

```
## Graficar resultados
```



Interpretación de resultados

UMAP permitió reducir las 30 variables originales del dataset a dos dimensiones preservando tanto estructura local como parte de la estructura global.

En la visualización se observa una separación clara entre los diagnósticos malignos (M) y benignos (B), similar a t-SNE. Sin embargo, UMAP tiende a generar agrupaciones más compactas y estructuradas.

A diferencia de t-SNE, UMAP mantiene mejor la relación global entre grupos, lo cual permite observar la distancia relativa entre clusters.

Esto confirma que el dataset presenta una estructura bien definida y que las variables originales contienen suficiente información para diferenciar ambos diagnósticos.

DESCRIPCIÓN TEÓRICA

2.3 UMAP (Uniform Manifold Approximation and Projection)

Descripción teórica

UMAP es un algoritmo de reducción de dimensionalidad no lineal basado en teoría de variedades y topología algebraica.

Su objetivo es:

- Preservar estructura local.
- Mantener parte de la estructura global.
- Reducir dimensionalidad de forma eficiente.

El algoritmo construye un grafo de vecinos más cercanos en alta dimensión y luego optimiza una representación en baja dimensión que preserve esa estructura.

Características principales

- Más rápido que t-SNE.
- Mejor preservación global.
- Escalable a grandes datasets.
- Sensible a parámetros como `n_neighbors` y `min_dist`.

Diferencias con t-SNE

- UMAP preserva mejor estructura global.
- UMAP es más rápido.
- t-SNE está más enfocado exclusivamente en estructura local.

USOS Y APLICACIONES

Usos y aplicaciones

UMAP se utiliza en:

1. Visualización de datos biomédicos.
2. Análisis de imágenes y visión por computadora.
3. Procesamiento de texto y embeddings.
4. Exploración de clusters en datasets de alta dimensión.

Es ampliamente utilizado en bioinformática y análisis genómico debido a su eficiencia y capacidad de preservación estructural.

2.4 ICA

Simular señales mezcladas (tipo EEG)

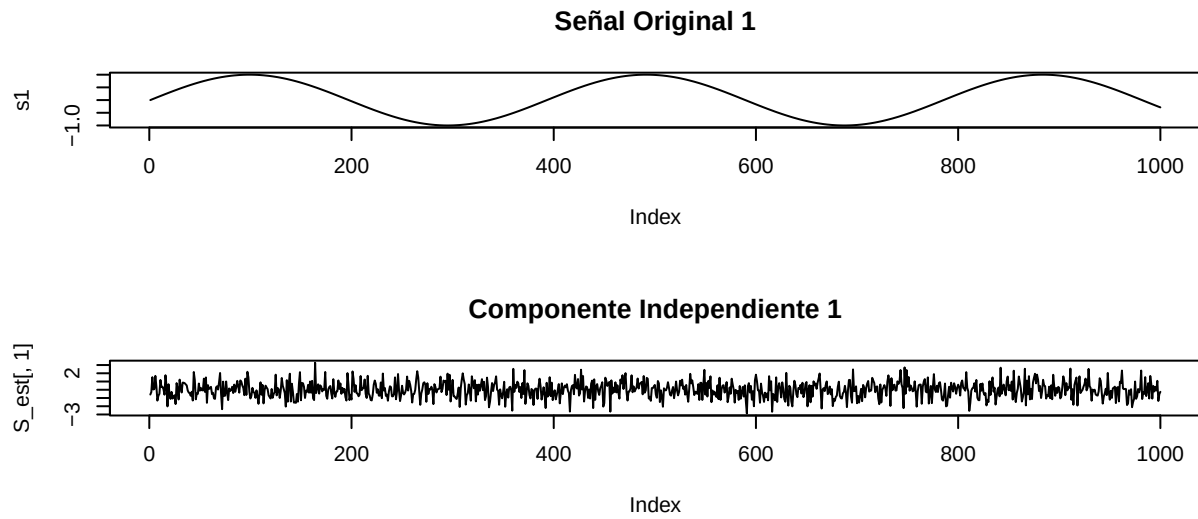
ICA se usa para separar señales mezcladas.

```
##           [,1]      [,2]      [,3]
## [1,] -0.8407135 -0.5604756 -1.1209513
## [2,]  0.1707491  1.7858378  0.5556604
## [3,]  2.8700890  3.5907349  4.1494432
## [4,]  0.6537921  2.1185380  1.1890463
## [5,]  0.7579519  2.1933080  1.3225957
## [6,]  3.1525920  3.7950595  4.5101245
```

Aplicar ICA

```
##           [,1]      [,2]      [,3]
## [1,] -0.58264046 -0.04773744  0.1574694
## [2,] -0.27525356  0.95943651  0.2006183
## [3,]  1.52925252  0.98301566  0.2317356
## [4,]  0.02860304  0.96214217  0.1632056
## [5,]  0.08818315  0.96225366  0.1417386
## [6,]  1.68784028  0.98307927  0.1667329
```

Graficar señales originales vs recuperadas



INTERPRETACIÓN

Interpretación de resultados

El algoritmo ICA permitió separar las señales mezcladas en componentes estadísticamente independientes.

A diferencia de PCA o SVD, ICA no busca maximizar varianza sino maximizar independencia estadística entre componentes.

En el experimento se generaron tres señales independientes que fueron mezcladas linealmente. ICA logró recuperar componentes similares a las señales originales, demostrando su capacidad para separar fuentes ocultas.

Este tipo de técnica es ampliamente utilizada en:

- Análisis de señales EEG
- Eliminación de artefactos en señales biomédicas
- Separación de voz en procesamiento de audio

ICA es especialmente útil cuando se asume que las señales observadas son combinaciones lineales de fuentes independientes.

DESCRIPCIÓN TEÓRICA

2.4 ICA (Independent Component Analysis)

Descripción teórica

ICA es un algoritmo de separación de fuentes que busca encontrar componentes estadísticamente independientes a partir de señales observadas.

El modelo matemático asume:

$$\text{begin : math : display } X = AS \text{ end : math : display}$$

donde:

- X son las señales observadas
- A es la matriz de mezcla desconocida
- S son las fuentes independientes

El objetivo de ICA es estimar S a partir de X sin conocer A.

Características principales

- Busca independencia estadística.
- Funciona mejor cuando las señales no son gaussianas.
- Es un método no supervisado.
- Muy utilizado en procesamiento de señales.

Diferencias con PCA

- PCA maximiza varianza.
- ICA maximiza independencia.
- PCA produce componentes ortogonales.
- ICA produce componentes estadísticamente independientes.

Comparación general entre algoritmos

Pega esto como texto en tu informe:

3. Comparación general entre algoritmos

Los cuatro algoritmos analizados pertenecen al aprendizaje no supervisado, pero tienen objetivos y aplicaciones distintas.

SVD

SVD es un método lineal basado en álgebra lineal que permite descomponer matrices y reducir dimensionalidad. Es especialmente útil en sistemas de recomendación y análisis de factores latentes. Preserva la estructura global de los datos y permite reconstrucción aproximada de la matriz original.

Ventajas: • Base matemática sólida. • Permite reconstrucción. • Interpretación clara de valores singulares.

Limitaciones: • Método lineal. • No captura relaciones no lineales complejas.

t-SNE

t-SNE es un método no lineal enfocado en visualización. Preserva principalmente la estructura local de los datos.

Ventajas: • Excelente para visualización. • Detecta clusters complejos. • Funciona bien con alta dimensionalidad.

Limitaciones: • No preserva bien estructura global. • Sensible a parámetros. • No permite reconstrucción.

UMAP

UMAP también es un método no lineal, pero preserva tanto estructura local como parte de la global.

Ventajas: • Más rápido que t-SNE. • Mejor preservación global. • Escalable a datasets grandes.

Limitaciones: • Sensible a parámetros. • No produce componentes interpretables.

ICA

ICA es un método de separación de fuentes que busca independencia estadística entre componentes.

Ventajas: • Ideal para señales biomédicas. • Permite separar fuentes ocultas. • Muy útil en procesamiento de señales.

Limitaciones: • Requiere supuestos de independencia. • Sensible a ruido. • No está diseñado principalmente para visualización.

Table 1: Comparación entre algoritmos

Algoritmo	Tipo	Naturaleza	Objetivo
SVD	Factorización	Lineal	Reducción y recomendación
t-SNE	Reducción	No lineal	Visualización
UMAP	Reducción	No lineal	Visualización eficiente
ICA	Separación	Lineal (estadístico)	Separar fuentes independientes

Conclusiones

En esta investigación se analizaron y aplicaron cuatro algoritmos fundamentales de aprendizaje no supervisado: SVD, t-SNE, UMAP e ICA.

Se observó que los métodos lineales como SVD permiten identificar factores latentes y reducir dimensionalidad de forma estructurada, siendo especialmente útiles en sistemas de recomendación.

Por otro lado, los métodos no lineales como t-SNE y UMAP demostraron ser herramientas poderosas para la visualización de datos de alta dimensión, permitiendo identificar agrupamientos naturales en el dataset de cáncer de mama.

Finalmente, ICA mostró su utilidad en la separación de señales biomédicas, destacando su importancia en aplicaciones como EEG y procesamiento de audio.

Uno de los principales aprendizajes fue comprender que no todos los algoritmos de reducción de dimensionalidad cumplen el mismo propósito; algunos priorizan varianza global, otros estructura local, y otros independencia estadística.

Entre las dificultades encontradas se incluyen la selección adecuada de parámetros y la interpretación correcta de las representaciones obtenidas.

En general, esta actividad permitió comprender la importancia del aprendizaje no supervisado en el análisis moderno de datos y su aplicación en problemas reales.